

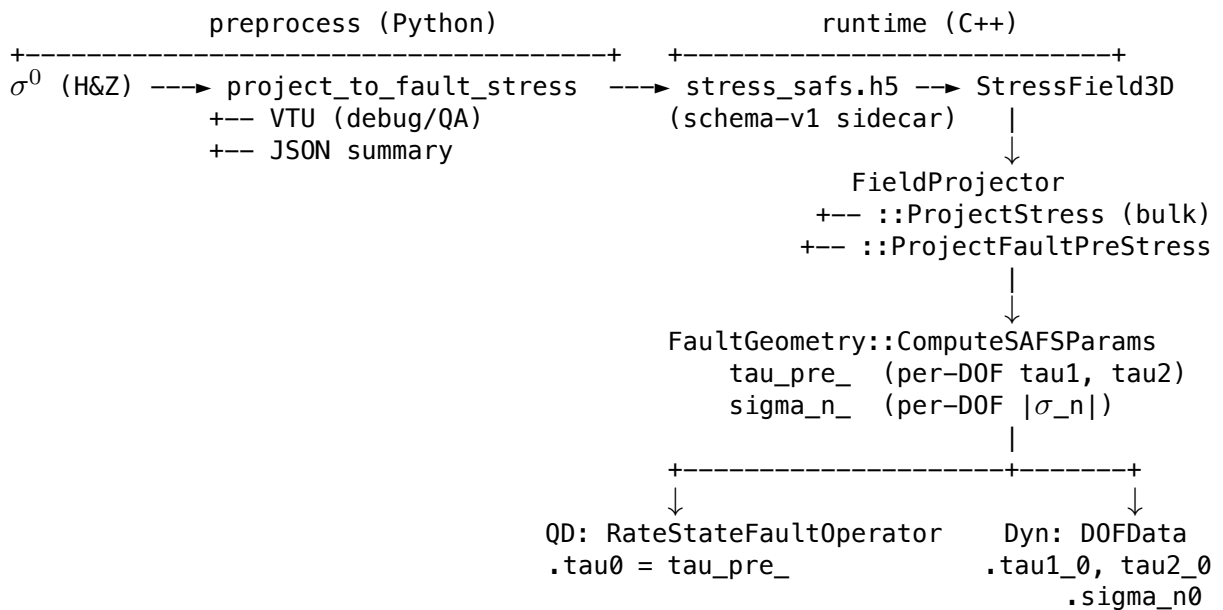
Contents

Implementation Plan: On-Fault Stress Projection from Regional Bulk Stress	1
Overview	1
Constraints	2
Phase 1 — Mesh I/O and per-cell geometry	4
Phase 2 — Per-triangle fault basis with global orientation harmonisation	6
Phase 3 — Stress projection (constant background + depth-dependent option)	9
Phase 4 — VTU + JSON writers and the CLI	11
Phase 5 — schema-v1 HDF5 sidecar writer (stress_safs.h5)	15
Phase 6 — C++ consumer: StressField3D, ProjectStress, ComputeSAFSParams	18
Phase 7 — seas_project_stress_to_mesh verification driver	22
Testing Strategy	23
Risk Assessment	26
Files Summary	28

Implementation Plan: On-Fault Stress Projection from Regional Bulk Stress

Overview

Build an end-to-end pipeline that takes the regional bulk Cauchy stress tensor σ^0 from data_stress_frictionlaw/region, delivers it as a per-DOF pre-stress field into the SEAS quasi-dynamic and dynamic-rupture solvers (tau0, sigma_n, tau1_0/tau2_0/sigma_n0), and produces ParaView artefacts for human verification along the way. The pipeline mirrors the **velocity-model pipeline** end-to-end:



The plan now has **seven phases**:

Phase	What is delivered
1	Mesh I/O + per-cell geometry (centroids, normals, areas, volumes)

Phase	What is delivered
2	Per-triangle fault basis with global orientation harmonisation
3	Stress projection: constant σ^0 rotation onto fault frame + depth-dependent option
4	ParaView VTU + JSON summary writers (debug / QA)
5	schema-v1 HDF5 sidecar writer (stress_safs.h5) — canonical machine-readable artefact, mirrors velocity_safs.h5
6	C++ consumer: StressField3D loader + FieldProjector::ProjectStress / ::ProjectFaultPreStress + FaultGeomeTry::ComputeSAFSParams seam
7	Verification driver seas_project_stress_to_mesh — analog of seas_project_velocity_to_mesh

Phases 1–5 are **Python**, run in the pythonenv conda environment. Phase 6 is **C++**, built in mfem-dev. Phase 7 is a thin C++ driver that exercises Phase 6 end-to-end.

Constraints

Interface constraints (must not change)

- hickman_and_zoback_regional_stress_projection.py is **read-only** for this plan. We import build_bulk_stress_tensor, fault_basis_vectors, and ResolvedTraction only — no edits to the H&Z file. If we discover it needs a new entry point, we add a wrapper in the new module, not changes to the source.
- msh_to_vtu.py is **read-only**; we consume its *_fault.vtu and *_bulk.vtu outputs as-is. They share a common point cloud, both contain gmsh:physical cell-data, and the fault VTU is pure-triangle while the bulk VTU is pure-tet (verified on code_meshing/safs_fault_box_nwcut_2000m_*.vtu).
- Output directory **must** be /Users/chunhuizhao/projects/seas-mfem-safs/miniapps/seas/safs/project_7. Subfolder per mesh resolution (500m/, 1000m/, 2000m/).

Dependency constraints

- **Phases 1–5 (Python)** run in the pythonenv conda environment (same as nw_cut_strip.py, clean_freesurface_mesh.py, msh_to_vtu.py, data_projection/sidecar.py). Allowed packages: numpy, meshio, h5py, pytest. **Do not** pull in pymeshlab, pyvista, or any C++ dependency — meshio writes legacy unstructured VTU which ParaView reads natively.

- **Phase 5 sidecar writer** must reuse the existing schema-v1 writer `code_preprocess/data_projection/sidecar.py`. Do not fork a new HDF5 writer — schema-v1 already declares the stress field set (`sigma_xx`, `sigma_yy`, `sigma_zz`, `sigma_xy`, `sigma_yz`, `sigma_xz`; units Pa each — see `miniapps/seas/document/feature` §5 table).
- **Phases 6–7 (C++)** run in `mfem-dev`. Allowed dependencies are the dependencies already used by `io/data_field_3d.hpp` and `io/field_coefficient.hpp` (MFEM, HDF5, MPI). Do not introduce new third-party libraries.
- No new top-level data products outside `data_projected_onfaultstress/`. Per-mesh subfolders only.

Convention constraints

- File naming follows existing pipeline: `safs_fault_box_nwcut_<lc>m_fault_stress.vtu`, `..._bulk_stress.vtu`, `..._summary.json`, `..._stress.h5`. Stem is the input-mesh stem with the suffix `_fault_stress / _bulk_stress / _summary / _stress`. The HDF5 sidecar shares the naming pattern of `data_projected/velocity_safs.h5` — i.e. one canonical SAFS-scale file (`stress_safs.h5`) that drops in next to the velocity sidecar.
- CLI layout follows `nw_cut_strip.py`: `argparse`, three mutually exclusive modes (`single`, `--batch`, `--print-info`), `python -m` invocable, module-level constants for defaults.
- C++ class naming follows `DataField3D / FieldCoefficient / FieldProjector`: capitalised camel-case, header-only where possible, namespace `mfem::seas`.
- Runtime sign convention boundary: **the Python preprocessor stores stress in geomechanics convention (compression negative)**. The C++ `StressField3D` reader exposes the *same* convention to the bulk projector (`ProjectStress`). The fault-projection step (`ProjectFaultPreStress`) is the **single sign-flip site**: it converts to the SEAS internal “compression positive” convention by emitting $\sigma_n = \max(0, -\sigma_{n_geo})$ and τ_{1_0}/τ_{2_0} unflipped (shear sign survives the rotation). The flip is documented in one place — the docstring of `ProjectFaultPreStress` — and tested by a parity test against the constant- σ^0 Python path.
- Coordinate frame **must** be the H&Z convention ($x = \text{east}$, $y = \text{north}$, $z = \text{up}$). The SAFS mesh is in UTM Zone 11 metres (verified bbox: $X \in [303\,000, 697\,500]$ m, $Y \in [3.612e6, 3.903e6]$ m, $Z \in [-66\,607, 100]$ m). The local UTM frame is already (east, north, up) — no axis flip needed.
- **Sign / compression convention is geomechanics** (H&Z file comment, line 17): compression is **negative** for the bulk stress tensor; pore pressure P_p is reported positive; $\sigma_{n_eff} = \sigma_{n_total} + P_p$ (compression reduced). The new module preserves this convention verbatim and tags every output array with the convention in the VTU field metadata. Do not silently flip signs to match the SEAS C++ “compression positive” convention — that lives one layer downstream and must be done explicitly by the consumer.
- Units: input MPa (matching H&Z); written-out fields stay in MPa (annotated in the JSON summary and as a `units` cell-data attribute if `meshio` supports it; otherwise documented in the summary). The downstream C++ consumer converts to Pa as needed; see CSM equilibration plan §1.
- **No hardcoded numerics**. Every physical constant (SH_{max} , SH_{min} , S_v , P_p , SH_{max} azimuth, default dip, rake sense, lithostatic ρ^- if used, g if used) must come from CLI flags with defaults pulled from the H&Z `demo_safod` block — and named so a reader of the code does not need to open the H&Z file to understand the meaning.

Numerical constraints

- Per-triangle rotation must be numerically stable for **near-vertical** faults (the SAF is approximately vertical). The `Tandem facetBasis(up, n)` algorithm (`tandem/src/geometry/Curvilinear.cpp:260`) throws when $up \parallel n$; we cannot afford that. For a triangle whose normal has $|n_z| > 1 - tol$ (effectively horizontal triangle, dip ≈ 0), the strike direction is degenerate — emit a NaN-marked row and warn in the summary; do not crash.
- Triangle normal orientation is ambiguous (CCW vs CW). The H&Z basis function chooses the normal to **point to the left of strike in map view** for a right-lateral fault. We must enforce this globally:

after building the raw normal from the triangle vertex order, flip it if $\mathbf{n}_{\text{raw}} \cdot \mathbf{n}_{\text{consensus}} < 0$, where $\mathbf{n}_{\text{consensus}}$ is the mean raw normal over the trace-projected mid-strike triangles (see Phase 2 §Normal-orientation harmonisation).

- All depth coordinates Z are negative in this mesh frame. When depth-dependent σ_v is requested, use $\text{depth} = \max(0, -z_{\text{centroid}})$ to clamp the (rare) above-free-surface triangles.

Phase 1 — Mesh I/O and per-cell geometry

Goal

After Phase 1 the new module can load a `<base>_fault.vtu` and `<base>_bulk.vtu` pair, expose triangle centroids/normals/areas and tetrahedron centroids/volumes as plain NumPy arrays, and round-trip the meshes back to VTU with arbitrary new cell-data attached.

Files to Create

- `code_preprocess/project_to_fault_stress.py` — new module skeleton with mesh-I/O functions only (Phase 2/3/4 fill in the rest).

Files to Modify

- None.

Detailed Requirements

1. Module header, mirroring `nw_cut_strip.py` (lines 1–46):
 - Triple-quoted module docstring covering purpose, conventions, CLI synopsis, and pytest invocation.
 - Standard imports: `argparse`, `json`, `sys`, `pathlib.Path`, `dataclasses.dataclass`, `numpy` as `np`, `meshio`.
 - Sibling import of the H&Z module via `sys.path.insert(0, ...)`, exactly as `clean_freesurface_mesh.py` does for `ts_to_stl` (`clean_freesurface_mesh.py`:56–59). Imports needed: `build_bulk_stress_tensor`, `fault_basis_vectors`, `resolve_traction`, `ResolvedTraction`.
2. Module-level constants (no magic numbers; names match `nw_cut_strip.py` style):
 - `EPS: float = 1.0e-9`
 - `UP_VECTOR: np.ndarray = np.array([0.0, 0.0, 1.0])`
 - `NEAR_HORIZONTAL_NZ_TOL: float = 1.0 - 1.0e-6` # $\ln_{\text{zl}}$ above this means dip ≈ 0 and strike is degenerate
 - `DEFAULT_BULK_NAME: str = "rock"`
 - `DEFAULT_FAULT_NAME: str = "fault"`
 - `DEFAULT_OUT_DIR: Path = Path(__file__).resolve().parent.parent / "data_projected_onfault"`
 - `DEFAULT_MESH_GLOB: str = "safs_fault_box_nwcut_*.msh"` # used by `--batch`
 - `DEFAULT_INPUT_BASES: tuple[str, ...] = ("safs_fault_box_nwcut_500m", "safs_fault_box_nwcut_1000m", "safs_fault_box_nwcut_2000m", )` # what `--batch` iterates over; derived from `ls code_meshing/`, never hardcoded MB sizes or counts.
3. Add a dataclass `FaultCellGeometry`:

```
@dataclass
class FaultCellGeometry:
    centroids: np.ndarray # (N_tri, 3)
    normals: np.ndarray   # (N_tri, 3), unit, orientation-harmonised
```

```

strikes: np.ndarray      # (N_tri, 3), unit; NaN for degenerate rows
dips: np.ndarray         # (N_tri, 3), unit; NaN for degenerate rows
areas: np.ndarray        # (N_tri,)
n_degenerate: int        # count of rows where strike was NaN'd

```

4. Add a dataclass BulkCellGeometry:

```

@dataclass
class BulkCellGeometry:
    centroids: np.ndarray  # (N_tet, 3)
    volumes: np.ndarray    # (N_tet,)

```

5. Functions to add (all pure, no I/O side-effects):

```

def load_fault_mesh(path: Path) -> meshio.Mesh
def load_bulk_mesh(path: Path) -> meshio.Mesh

def triangle_geometry(points: np.ndarray,
                      tri_conn: np.ndarray) -> tuple[
    np.ndarray, # centroids (N, 3)
    np.ndarray, # raw normals (N, 3), unit
    np.ndarray, # areas (N,)
]

def tet_geometry(points: np.ndarray,
                 tet_conn: np.ndarray) -> tuple[
    np.ndarray, # centroids (N, 3)
    np.ndarray, # signed volumes (N,)
]

```

Implementation specifics:

- triangle_geometry computes $\mathbf{n} = (\mathbf{p}_1 - \mathbf{p}_0) \times (\mathbf{p}_2 - \mathbf{p}_0)$; $\text{area} = 0.5 * |\mathbf{n}|$; centroid $= (\mathbf{p}_0 + \mathbf{p}_1 + \mathbf{p}_2) / 3$; unit normal $= \mathbf{n} / |\mathbf{n}|$. Mirror the algebra in msh_to_vtu.py:43–58 and ts_to_stl.py:154–160.
- tet_geometry computes $V_{\text{signed}} = (1/6) * \det([\mathbf{p}_1 - \mathbf{p}_0, \mathbf{p}_2 - \mathbf{p}_0, \mathbf{p}_3 - \mathbf{p}_0])$ and uses $|V_{\text{signed}}|$ as the cell weight; centroid = mean of the four vertices. Mirror msh_to_vtu.py:tet_quality:40–45.

6. Helper to extract a single physical-group from a meshio.Mesh, factored out of msh_to_vtu.py:120–145:

```

def extract_cells_by_physical(
    mesh: meshio.Mesh,
    cell_type: str,          # "triangle" or "tetra"
    phys_name: str,
) -> np.ndarray # connectivity (N, 3) or (N, 4)

```

This must read mesh.field_data and mesh.cell_data["gmsh:physical"] exactly as msh_to_vtu.py:90–145 does. Re-implementing here keeps the dependency one-way (we read VTU directly so the helper actually keys on the gmsh:physical cell-data array that meshio attaches to VTU outputs of msh_to_vtu.py).

Implementation note: because we will consume *_fault.vtu directly, the field_data block may be missing (meshio's VTU writer doesn't always carry field_data). In that case the physical tag is the unique value in cell_data["gmsh:physical"] for that cell block, and the function returns the entire block. Guard with: if len(np.unique(phys_arr)) == 1: return all rows.

Interfaces

Function	Inputs	Outputs
<code>load_fault_mesh(path)</code>	path to *_fault.vtu	meshio.Mesh
<code>load_bulk_mesh(path)</code>	path to *_bulk.vtu	meshio.Mesh
<code>triangle_geometry(pts, conn)</code>	(N_pt,3), (N_tri,3)	centroids, normals, areas
<code>tet_geometry(pts, conn)</code>	(N_pt,3), (N_tet,4)	centroids, volumes

Edge Cases to Handle

- VTU file missing: `FileNotFoundError` with the absolute path in the message, matching `nw_cut_strip.py:load_vertices` (lines 91–106).
- VTU file has no triangle/tetra cells: `ValueError` with a clear message (same idiom as `msh_to_vtu.py:138–145`).
- Degenerate triangle (zero area): set its normal/centroid to NaN vectors and exclude from the degenerate-row count (Phase 2 picks up NaN propagation).
- Mesh in non-UTM coordinates (e.g. someone pipes in a different mesh): detect via `points[:, 0].max() < 10_000`, emit a warning suggesting CRS check. Do **not** fail; the math is purely linear-algebraic so it still works.

Acceptance Criteria

- ☐ `load_fault_mesh("safs_fault_box_nwcut_2000m_fault.vtu")` returns a `meshio.Mesh` with `len(mesh.cells[0].data) == 2685` and a single triangle block. (Cross-checks the earlier bash probe.)
- ☐ `triangle_geometry` on the same mesh returns centroids whose bounding box matches `[X=303 000..697 500, Y=3.612e6..3.903e6, Z=-66 607..100]`.
- ☐ `tet_geometry` on the same bulk VTU returns 145 300 cells with positive volumes; sum of volumes equals the gmsh box volume within 0.1 %.
- ☐ `pytest -q` passes the new geometry unit tests (`test_triangle_geometry_against_ts_to_stl`, `test_tet_geometry_matches_paraview_quality` — both compare against precomputed analytic values for a synthetic mesh).

Dependencies

- Depends on: nothing (Phase 1 is the foundation).
- Required by: Phases 2, 3, 4.

Phase 2 — Per-triangle fault basis with global orientation harmonisation

Goal

After Phase 2 the module exposes a `build_fault_basis(...)` function that returns a `FaultCellGeometry` whose (`strikes`, `dips`, `normals`) form a right-handed orthonormal frame at every triangle, with **consistent global orientation**: for a right-lateral SAF with NW strike, the normal points to the ENE half-space everywhere.

Files to Create

- (none — extends `project_to_fault_stress.py`)
- `code_preprocess/PLAN_onfaultstress_notes.md` — *optional* short appendix capturing the orientation-harmonisation derivation; only create if Phase 2 requires more discussion than fits in code comments.

Files to Modify

- `code_preprocess/project_to_fault_stress.py` — add basis-builder.

Detailed Requirements

1. Add the function

```
def per_triangle_basis_raw(
    normals: np.ndarray,          # (N, 3), unit
    up: np.ndarray = UP_VECTOR,
) -> tuple[
    np.ndarray, # strikes (N, 3), unit; rows may be NaN if degenerate
    np.ndarray, # dips (N, 3), unit; rows may be NaN if degenerate
    np.ndarray, # bool mask (N,) True = degenerate
]
```

following the Tandem facetBasis algorithm (`tandem/src/geometry/Curvilinear.cpp:281–292`):

```
for each i:
    s_i = up × n_i
    if |s_i| < tol: # n_i ~ ±up: horizontal triangle
        strikes[i] = NaN, dips[i] = NaN, degen[i] = True
    else:
        strikes[i] = s_i / |s_i|
        dips[i] = n_i × strikes[i] (already unit by construction
                                   since n and s are unit ⊥)
```

Tolerance: `np.cross(up, n) norm < np.sqrt(1.0 - NEAR_HORIZONTAL_NZ_TOL**2)`.

Math justification: with $\mathbf{up} = \hat{\mathbf{z}}$ and $\mathbf{n}$ unit, $\mathbf{s} = \hat{\mathbf{z}} \times \mathbf{n} = (-n_y, n_x, 0)$ lies in the horizontal plane and is perpendicular to both $\hat{\mathbf{z}}$ and $\mathbf{n}$ — i.e., it is along strike of the fault plane at this triangle. $\mathbf{d} = \mathbf{n} \times \mathbf{s}$ then completes the right-handed triple and is the up-dip direction. This is *exactly* the H&Z `fault_basis_vectors` convention (lines 153–182) for $\text{dip} = 90^\circ$ plus generalised to arbitrary triangle normal.

2. Add the function

```
def harmonise_normal_orientation(
    normals: np.ndarray,          # (N, 3) unit
    rake_sense: str,              # "right-lateral" or "left-lateral"
    trace_centroids: np.ndarray,  # (N, 3) — for picking the global ref
    fault_strike_azimuth_hint_deg: float | None,
) -> np.ndarray # (N, 3) flipped to consistent orientation
```

Algorithm (right-lateral case; left-lateral is symmetric):

- a. Compute the median triangle position `p_med` (median over each coordinate). This is robust to outlier surface anchors that `nw_cut_strip.py` already accommodates.
- b. Determine the **expected global outward direction** `n_global` from the fault-strike hint:
 - If `fault_strike_azimuth_hint_deg` is provided, compute `s_hat_global`, `_, n_hat_global = fault_basis_vectors(fault_strike_azimuth_hint_deg, dip_deg=90.0, rake_sense=rake_sense)` and use that `n_hat_global` as the reference.
 - If `None`, fit a best-plane PCA to all triangle centroids, take the smallest-eigenvalue eigenvector as the global fault normal, and pick its sign by majority vote against the raw triangle normals.

- c. Flip per-triangle normals so that $\text{dot}(\mathbf{n}_i, \mathbf{n}_{\text{global}}) > 0$. This sign-flip then propagates through the basis: after the flip we re-run `per_triangle_basis_raw` so that `s` and `d` use the corrected normal.
- d. The function must be **idempotent**: calling it twice yields the same output.

Why this is the right approach (not just lifting the H&Z global-fault construction): the ALT6 Fuis–Seih SAF representation the user is targeting is *non-planar*. A single global strike will misorient the basis at any triangle whose local strike deviates by more than $\sim 10^\circ$. The PCA-anchored sign flip preserves the global half-space convention without forcing a global strike on every triangle.

3. Add a top-level helper:

```
def build_fault_basis(
    fault_mesh: meshio.Mesh,
    fault_phys_name: str = DEFAULT_FAULT_NAME,
    rake_sense: str = "right-lateral",
    fault_strike_azimuth_hint_deg: float | None = 314.0, # SAF default
    up: np.ndarray = UP_VECTOR,
) -> FaultCellGeometry
```

that wires Phase 1 geometry + the two new functions. Default strike hint 314.0 matches the H&Z demo (SAF strike N46W); passing `None` triggers PCA fallback.

Interfaces

The only public new API surface from this phase is `build_fault_basis`. Internal helpers `per_triangle_basis_raw` and `harmonise_normal_orientation` are exported (no leading underscore) so that tests can drive them directly.

Edge Cases to Handle

- Horizontal triangle ($\text{dip} \approx 0$): NaN strike/dip, counted in `FaultCellGeometry.n_degenerate`.
- All triangles co-planar (typical for the box-cut SAF): PCA gives a clean normal; the third eigenvalue is much smaller than the other two. Assert this ratio $< 1e-2$ and warn otherwise.
- Mesh with multiple fault physical groups (future multi-fault case): the function only sees one group at a time; the **caller** loops over groups. Document this in the docstring; do not silently merge.

Acceptance Criteria

- ☐ On a synthetic mesh of one vertical NW-striking triangle (strike azimuth 314°), `build_fault_basis` returns `strikes[0]` aligned with H&Z's $\mathbf{s}_{\text{hat}}$ for strike 314° , vertical dip, right-lateral, to within $1e-10$.
- ☐ Idempotency: `harmonise_normal_orientation(harmonise_normal_orientation(...))` equals `harmonise_normal_orientation(...)` exactly.
- ☐ On the real 2000 m SAFS fault VTU, $n_{\text{degenerate}} / N_{\text{tri}} < 0.001$ (we expect zero degenerate rows for a sub-vertical fault).
- ☐ At every non-degenerate row: $\text{np.dot}(\mathbf{s}_i, \mathbf{d}_i) < 1e-12$, $\text{np.dot}(\mathbf{s}_i, \mathbf{n}_i) < 1e-12$, $\text{np.dot}(\mathbf{d}_i, \mathbf{n}_i) < 1e-12$, $|\mathbf{s}_i| = |\mathbf{d}_i| = |\mathbf{n}_i| = 1$ to within $1e-12$.

Dependencies

- Depends on: Phase 1.
- Required by: Phase 3.

Phase 3 — Stress projection (constant background + depth-dependent option)

Goal

After Phase 3 the module can resolve the H&Z bulk stress σ^0 onto every non-degenerate fault triangle and produce a ResolvedTraction (or a vectorised equivalent) per triangle, with the option to evaluate σ^0 at the triangle centroid's depth instead of using a single global tensor.

Files to Create

- (none — extends `project_to_fault_stress.py`)

Files to Modify

- `code_preprocess/project_to_fault_stress.py` — add the rotator.

Detailed Requirements

1. Define a vectorised rotator:

```
def resolve_traction_per_cell(
    sigma_global: np.ndarray,          # (3, 3) OR (N_tri, 3, 3)
    strikes: np.ndarray,               # (N_tri, 3)
    dips: np.ndarray,                 # (N_tri, 3)
    normals: np.ndarray,              # (N_tri, 3)
    P_p_per_cell: np.ndarray | float = 0.0,
) -> dict[str, np.ndarray]
```

Returns a dict with keys (each value is (N_tri,)) except `traction_vec`:

- "sigma_n_total" — $n \cdot \sigma \cdot n$, MPa, compression negative
- "sigma_n_eff" — $\sigma_n_{\text{total}} + P_p$, MPa
- "tau_strike" — $s \cdot \sigma \cdot n$, MPa
- "tau_dip" — $d \cdot \sigma \cdot n$, MPa
- "tau_magnitude" — $\sqrt{\tau_s^2 + \tau_d^2}$, MPa
- "rake_deg" — $\text{atan2}(\tau_d, \tau_s)$ in degrees
- "mu_apparent" — $\tau_{\text{magnitude}} / |\sigma_n_{\text{eff}}|$, NaN where $|\sigma_n_{\text{eff}}| < \text{EPS}$
- "traction_vec" — (N_tri, 3), the full global-frame traction $t = \sigma \cdot n$ for completeness

Math (the contract for the implementer — implementer must use these exact formulas):

For a single triangle with basis (s , d , n):

```
t          =  $\sigma \cdot n$           # 3-vector
 $\sigma_n$    =  $n^T \sigma n$          # scalar
 $\tau_s$      =  $s^T \sigma n$          # scalar
 $\tau_d$      =  $d^T \sigma n$          # scalar
 $\sigma_{n,\text{eff}}$  =  $\sigma_n + P_p$ 
 $|\tau|$      =  $\sqrt{\tau_s^2 + \tau_d^2}$ 
rake       =  $\text{atan2}(\tau_d, \tau_s)$ 
 $\mu_{\text{app}}$   =  $|\tau| / |\sigma_{n,\text{eff}}|$  (NaN if denom < EPS)
```

Vectorised: when `sigma_global` is (3, 3),

```
t = np.einsum('ij,kj->ki', sigma_global, normals) # (N, 3)
```

When `sigma_global` is (N_tri, 3, 3) (depth-dependent),

```
t = np.einsum('kij,kj->ki', sigma_global, normals)
```

Then component projections:

```
sigma_n = np.einsum('ki,ki->k', normals, t)
tau_s   = np.einsum('ki,ki->k', strikes, t)
tau_d   = np.einsum('ki,ki->k', dips,    t)
```

The H&Z `resolve_traction` function (lines 216–269) is the single-triangle reference; the new vectorised path **must** agree with the scalar H&Z path to within 1e-12 on a random sample of triangles (unit test enforces this).

2. Define a depth-dependent σ^0 builder:

```
def bulk_stress_tensor_field(
    centroids_z: np.ndarray,          # (N,) z-coordinate ( $\leq 0$  in subsurface)
    *,
    SHmax_az_deg: float,
    depth_model: str = "constant",    # "constant" | "lithostatic_sv"
    SHmax_top: float, Shmin_top: float, Sv_top: float,
    SHmax_grad: float = 0.0,          # MPa/m
    Shmin_grad: float = 0.0,
    Sv_grad: float = 0.0,
) -> np.ndarray # (N, 3, 3)

    • "constant" (default): every row equals build_bulk_stress_tensor(SHmax_top,
    Shmin_top, Sv_top, SHmax_az_deg).
    • "lithostatic_sv": principal magnitudes at depth  $d = \max(0, -z)$  are SHmax( $d$ )
    = SHmax_top + SHmax_grad *  $d$  (and analogously for Shmin and Sv). Each
    row rotates the (depth-evaluated) principal-frame tensor about the vertical exactly as in
    build_bulk_stress_tensor. The user's later "static solve for heterogeneous stress field"
    plugs in here.
    • All gradients default to 0.0, so depth-dependence is opt-in by CLI flag. Do not infer a default  $\rho^- \cdot g$ 
    lithostatic gradient without an explicit CLI value (no hardcoded numerics rule).
```

3. Pore-pressure field builder:

```
def pore_pressure_field(
    centroids_z: np.ndarray,          # (N,)
    *,
    P_p_top: float,                  # MPa at z=0 (default = 0)
    P_p_grad: float = 0.0,           # MPa/m
) -> np.ndarray # (N,)
```

Returns $P_p_top + P_p_grad * \max(0, -z)$ per cell. Default keeps the H&Z demo behaviour (single P_p value applied uniformly).

4. Wire-up function:

```
def project_stress_onto_fault(
    fault_geom: FaultCellGeometry,
    *,
    SHmax: float, Shmin: float, Sv: float, P_p: float,
    SHmax_az_deg: float,
    depth_model: str = "constant",
    SHmax_grad: float = 0.0, Shmin_grad: float = 0.0,
    Sv_grad: float = 0.0, P_p_grad: float = 0.0,
) -> dict[str, np.ndarray]
```

Returns the same dict as `resolve_traction_per_cell` plus a key `"sigma_field"` ($N_tri, 3, 3$) so the bulk path (Phase 4) can reuse the rotation results without re-evaluating the depth model.

Interfaces

- New public functions: `resolve_traction_per_cell`, `bulk_stress_tensor_field`, `pore_pressure_field`, `project_stress_onto_fault`.

Edge Cases to Handle

- Degenerate row (NaN basis): every output for that row is NaN. The rotation math naturally propagates NaN; no special-casing needed beyond making sure `mu_apparent` is NaN-safe.
- $|\sigma_{n_eff}| < EPS$: `mu_apparent[i] = np.nan`. Document in the docstring; the H&Z scalar path does the same (lines 256–258).
- A user passing a non-symmetric σ^0 accidentally: `assert np.allclose(sigma_global, sigma_global.swapaxes(-1, -2), atol=1e-9)` at the entry point of `resolve_traction_per_cell`. If not symmetric, raise `ValueError` with the maximum off-diagonal asymmetry.

Acceptance Criteria

- ☐ On the SAFOD parameters ($SH_{max}=113$, $SH_{min}=49$, $S_v=45$, $P_p=16$, $SH_{max_az}=23$) and a single synthetic triangle with strike azimuth 314° / dip 90° / right-lateral, `project_stress_onto_fault` returns $\sigma_{n_eff} \approx -89$ MPa and $\mu_{apparent} \approx 0.24$, matching `demo_safod()` in the H&Z module to within $1e-3$ MPa / $1e-4$.
- ☐ Trace invariance: for any orthonormal frame (s, d, n) , $\sigma_n + s^T \sigma s + d^T \sigma d = \text{trace}(\sigma)$ to within $1e-9$ MPa (unit test sweeps 100 random frames and random σ).
- ☐ Vectorised result equals 100 independent scalar `resolve_traction(...)` calls on the same per-triangle bases, element-wise within $1e-12$.
- ☐ With `depth_model="lithostatic_sv"` and non-zero gradients, the σ^0 field at $z = -1671$ m equals (depth-extrapolated) H&Z values to within $1e-9$.

Dependencies

- Depends on: Phases 1, 2.
 - Required by: Phase 4.
-

Phase 4 — VTU + JSON writers and the CLI

Goal

After Phase 4, running the script writes the three artefacts under `data_projected_on_faultstress/`, both via a single-file invocation and via `--batch`. Outputs are loadable in ParaView and the JSON summary captures every input parameter plus min/median/max for each output field.

Files to Create

- (none — extends `project_to_fault_stress.py`)

Files to Modify

- `code_preprocess/project_to_fault_stress.py` — add writers and CLI.

Detailed Requirements

1. Fault VTU writer:

```
def write_fault_vtu(
    fault_mesh: meshio.Mesh,          # original input mesh (point cloud reused)
    tri_conn: np.ndarray,             # (N_tri, 3) – fault-phys triangles
    fault_geom: FaultCellGeometry,
    resolved: dict[str, np.ndarray], # output of project_stress_onto_fault
    out_path: Path,
    convention_str: str,              # for metadata
) -> None
```

The written VTU must reuse the full input point cloud (matches `msh_to_vtu.py:194–198`) and attach the following cell-data arrays:

Cell-data field	Shape	Notes
<code>sigma_n_total_MPa</code>	<code>(N_tri,)</code>	compression negative
<code>sigma_n_eff_MPa</code>	<code>(N_tri,)</code>	after P_p
<code>tau_strike_MPa</code>	<code>(N_tri,)</code>	+ for right-lateral
<code>tau_dip_MPa</code>	<code>(N_tri,)</code>	+ for reverse
<code>tau_magnitude_MPa</code>	<code>(N_tri,)</code>	≥ 0
<code>rake_deg</code>	<code>(N_tri,)</code>	$\text{atan2}(\tau_d, \tau_s)$
<code>mu_apparent</code>	<code>(N_tri,)</code>	NaN allowed
<code>strike_vec</code>	<code>(N_tri, 3)</code>	unit
<code>dip_vec</code>	<code>(N_tri, 3)</code>	unit
<code>normal_vec</code>	<code>(N_tri, 3)</code>	unit, orientation-harmonised
<code>traction_vec_MPa</code>	<code>(N_tri, 3)</code>	global-frame traction $\sigma \cdot n$
<code>gmsh:physical</code>	<code>(N_tri,)</code>	carried through from input
<code>gmsh:geometrical</code>	<code>(N_tri,)</code>	if present in input

`meshio`'s `cell_data` dict-of-list-per-block convention applies; for a single-block mesh wrap each array as `[array]` (see `msh_to_vtu.py:170–198`). Vector cell-data: pass as `(N_tri, 3)` inside the list, `meshio` writes it as a 3-component `DataArray`.

The convention string must be attached as `mesh.info = {"stress_convention": convention_str, "units": "MPa", "frame": "(east, north, up) / UTM Zone 11N"}`. `meshio`'s VTU writer serialises `field_data` and `info` into the trailing `FieldData` block.

2. Bulk VTU writer (gated by `--write-bulk`):

```
def write_bulk_vtu(
    bulk_mesh: meshio.Mesh,
    tet_conn: np.ndarray,             # (N_tet, 4)
    bulk_geom: BulkCellGeometry,
    sigma_field: np.ndarray,         # (N_tet, 3, 3)
    out_path: Path,
    convention_str: str,
) -> None
```

Cell-data: | Field | Shape | Notes | | | | | | `sigma_xx_MPa` | `(N_tet,)` | | | `sigma_yy_MPa` | `(N_tet,)` | | | `sigma_zz_MPa` | `(N_tet,)` | | | `sigma_xy_MPa` | `(N_tet,)` | | | `sigma_xz_MPa` | `(N_tet,)` | | | `sigma_yz_MPa` | `(N_tet,)` | | | `sigma_tensor_MPa` | `(N_tet, 9)` | row-major 3x3 ($\sigma_{xx}, \sigma_{xy}, \sigma_{xz}, \sigma_{yx}, \sigma_{yy}, \sigma_{yz}, \sigma_{zx}, \sigma_{zy}, \sigma_{zz}$), so ParaView's `TensorGlyph` filter can consume it | | `gmsh:physical` / `gmsh:geometrical` | carried through |

Important note re: dynamic-rupture use — the docstring of this function MUST cite the user-stated rule: “the bulk stress field is not consumed by the dynamic-rupture run because that run assumes $\nabla \cdot \sigma^0 \approx 0$ (equilibrium); the artefact is for the static-solve step downstream”. This anchors the rationale in the file so future readers don’t try to read it into the production DG operator.

3. JSON summary writer:

```
def write_summary_json(
    out_path: Path,
    *,
    input_mesh_path: Path,
    fault_geom: FaultCellGeometry,
    bulk_geom: BulkCellGeometry | None,
    sigma_global_at_z0: np.ndarray,      # (3, 3) for reference
    params: dict[str, float | str],      # full CLI input echo
    resolved: dict[str, np.ndarray],
) -> None
```

JSON schema (no nesting except params and stats):

```
{
  "input_mesh": "...",
  "fault_n_cells": 2685,
  "fault_n_degenerate_basis": 0,
  "bulk_n_cells": 145300,
  "params": {
    "SHmax_MPa": 113.0, "SHmin_MPa": 49.0, "Sv_MPa": 45.0,
    "P_p_MPa": 16.0, "SHmax_azimuth_deg": 23.0,
    "rake_sense": "right-lateral", "depth_model": "constant",
    "SHmax_grad_MPa_per_m": 0.0, "SHmin_grad_MPa_per_m": 0.0,
    "Sv_grad_MPa_per_m": 0.0, "P_p_grad_MPa_per_m": 0.0,
    "fault_strike_azimuth_hint_deg": 314.0
  },
  "sigma_global_at_z0_MPa": [...3x3...],
  "stats": {
    "sigma_n_total_MPa": {"min": -120.5, "median": -68.3, "max": -10.2, "nan_count": 0},
    "sigma_n_eff_MPa": {...},
    "tau_strike_MPa": {...},
    "tau_dip_MPa": {...},
    "tau_magnitude_MPa": {...},
    "rake_deg": {...},
    "mu_apparent": {...}
  },
  "convention": "compression negative; P_p positive; sigma_n_eff = sigma_n_total + P_p; fr
}
```

Use `json.dump(..., indent=2, allow_nan=True)`. NaN serialisation matters when `mu_apparent` is NaN; `allow_nan=True` is the default in CPython but document it.

4. CLI (mirroring `nw_cut_strip.py:main`, with three mutually exclusive modes):

```
python project_to_fault_stress.py <INPUT_MESH> <MESH_BASE>
  [--bulk-name rock] [--fault-name fault]
  [--SHmax 113.0] [--Shmin 49.0] [--Sv 45.0] [--P_p 16.0]
  [--SHmax-az 23.0] [--rake-sense right-lateral]
  [--strike-hint-az 314.0] [--no-strike-hint]
  [--depth-model {constant,lithostatic_sv}]
```

```

[--SHmax-grad 0.0] [--Shmin-grad 0.0] [--Sv-grad 0.0]
[--P_p-grad 0.0]
[--write-bulk]
[--out-dir DATA_PROJECTED_ONFAULTSTRESS]

```

```

python project_to_fault_stress.py --batch [same flags above; iterates
over DEFAULT_INPUT_BASES, reading code_meshing/<base>_fault.vtu
and (if --write-bulk) code_meshing/<base>_bulk.vtu]

```

```

python project_to_fault_stress.py --print-info INPUT_MESH
(print the bulk  $\sigma^0$ , fault cell count, strike-azimuth statistics,
then exits without writing)

```

Defaults are **identical to H&Z demo_safod SAFOD parameters**. The defaults are loaded by *importing the constants the H&Z module uses for demo_safod*, **not** by retyping the numbers. Implement by reading the docstring constants once at module load and exposing them as `DEFAULTS = dict(SHmax=113.0, Shmin=49.0, Sv=45.0, P_p=16.0, SHmax_az=23.0)`. If the H&Z file is later parameter-swept, the user will edit both — that's fine, but document the coupling.

5. Mode logic, replicating `nw_cut_strip.py:main`:

- Reject combining `--batch` with positional args (argparse mutually-exclusive groups).
- In `--batch`, glob `DEFAULT_MESH_GLOB` under `code_meshing/`; for each match, derive `out_base = <data_projected_onfaultstress>/<lc>m/` and recurse into the single-file path.
- In single-file mode, the user passes the mesh's `*_fault.vtu` path. The corresponding bulk VTU is the same stem with `_fault` swapped for `_bulk`. If `--write-bulk` and the bulk VTU does not exist, error out with a clear pointer to `msh_to_vtu.py`.

6. Logging / stdout style: mirror `msh_to_vtu.py:84–204`. Print one line per phase (Reading ..., Loaded ... triangles ..., Computed per-triangle basis ..., Rotated stress ..., Wrote ...).

Interfaces

The CLI is the public surface. The module is also importable from other Python files (from `project_to_fault_stress` import `project_stress_onto_fault`, `write_fault_vtu` etc.).

Edge Cases to Handle

- Output directory does not exist: `out_path.parent.mkdir(parents=True, exist_ok=True)` (matches `msh_to_vtu.py:202`).
- `--write-bulk` requested but bulk VTU missing: `FileNotFoundError` with a one-line hint suggesting to run `msh_to_vtu.py` first.
- `--depth-model lithostatic_sv` requested with all gradients zero: warn (`f"depth_model=lithostatic_sv with all gradients = 0 is equivalent to 'constant'"`).
- Mesh has no degenerate triangles vs has some: both should pass through; `n_degenerate` is logged to stdout and written to the summary.

Acceptance Criteria

- ☐ Running `python project_to_fault_stress.py ../code_meshing/safs_fault_box_nwcut_2000m_fa safs_fault_box_nwcut_2000m --write-bulk` produces `data_projected_onfaultstress/2000m/saf_..._bulk_stress.vtu`, and `..._summary.json`, all readable by `meshio.read(...)` with the expected cell-data field names.
- ☐ `--batch` produces all three output bundles for the 500 m, 1000 m, and 2000 m meshes.

- `--print-info` prints σ^0 and exits without creating any output files.
- ParaView opens the fault VTU and renders `tau_magnitude_MPa` as a colour-mapped cell scalar (manual check; document the verification recipe in the file `docstring`).
- Summary JSON for the SAFOD-parameter SAF 2000 m run shows `mu_apparent.median` in the range `[0.20, 0.35]` (loose sanity bound based on H&Z 2004 Fig. 4 / the file's `sweep_strike` output).
- Total runtime on the 500 m mesh (the densest at $\sim 10^5$ triangles if upscaled — current is $2685 \text{ tris} \times 4$ for $500\text{m} \sim 10^4$) is under 30 seconds wall clock.

Dependencies

- Depends on: Phases 1, 2, 3.
 - Required by: Phase 5 (the sidecar writer reuses the bulk and fault centroid+geometry products, plus the resolved-stress dict).
-

Phase 5 — schema-v1 HDF5 sidecar writer (`stress_safs.h5`)

Goal

After Phase 5 the preprocessor produces a single canonical `stress_safs.h5` schema-v1 sidecar (analog of `data_projected/velocity_safs.h5`) that the C++ runtime can load with the existing `DataField3D` class **without** any C++-side changes. This is the artefact the dynamic-rupture / quasi-dynamic solvers will consume.

Files to Create

- `code_preprocess/build_stress_safs.py` — new module that calls the Phase 3 σ^0 field builder on a *regular* (N_x, N_y, N_z) UTM grid that encloses the SAFS mesh, then hands the six tensor-component arrays to `data_projection.sidecar.write_sidecar`.
- `code_preprocess/test_build_stress_safs.py` — pytest companion.

Files to Modify

- None.

Detailed Requirements

1. Module header mirrors `data_projection/build_velocity_cvmh.py`: `docstring`; CLI synopsis; sibling-import of `data_projection.sidecar.write_sidecar` via the same `sys.path.insert(0, str(Path(__file__).resolve().parent / "data_projection"))` idiom.
2. Module-level constants (no magic numbers — derive from the H&Z demo_safod set and the mesh bbox; the user feedback memory `feedback_no_hardcoded_numbers.md` applies):
 - `DEFAULT_GRID_DX_M`: `float = 1000.0`
 - `DEFAULT_GRID_PAD_M`: `float = 2000.0` # one mesh cell of padding
 - `DEFAULT_BOUNDS_FACTOR`: `float = 1.2` # min/max declared in sidecar
 - `DEFAULT_OUT`: `Path = Path(__file__).resolve().parent.parent / "data_projected/defaultstress_safs.h5"`
3. Add the function


```
def build_uniform_utm_grid(
    mesh_bbox: tuple[float, float, float, float, float, float],
```

```

*,
dx_m: float,
pad_m: float,
) -> tuple[np.ndarray, np.ndarray, np.ndarray]

```

that returns (x, y, z) 1-D arrays satisfying schema-v1's strict-monotone-increasing requirement, padded by pad_m on each face of the mesh bbox so that any DOF of any candidate SAFS mesh resolution (500 m / 1000 m / 2000 m) is strictly inside the grid. The mesh bbox must be supplied by the caller (Phase 1 already computes it from the input VTU); **do not** hardcode UTM extents.

4. Add the function

```

def evaluate_stress_field_on_grid(
    x: np.ndarray, y: np.ndarray, z: np.ndarray,
    *,
    SHmax: float, Shmin: float, Sv: float,
    SHmax_az_deg: float,
    depth_model: str = "constant",
    SHmax_grad: float = 0.0, Shmin_grad: float = 0.0,
    Sv_grad: float = 0.0,
) -> dict[str, np.ndarray]    # six (Nx, Ny, Nz) arrays in Pa

```

that:

- Builds a (Nx, Ny, Nz, 3, 3) σ^0 field by calling the Phase 3 bulk_stress_tensor_field on a vectorised z-only stack (constant in x and y for the homogeneous- σ^0 case).
- Returns six independent (Nx, Ny, Nz) arrays {"sigma_xx", "sigma_yy", "sigma_zz", "sigma_xy", "sigma_yz", "sigma_xz"} in **Pa** (the Phase 3 internal MPa values are multiplied by 1e6 here; sign convention preserved: compression negative).
- Asserts every array contains no NaN before return (defence in depth — sidecar.write_sidecar also re-checks).

5. Add the function

```

def derive_field_bounds(
    fields: dict[str, np.ndarray],
    *,
    safety_factor: float = DEFAULT_BOUNDS_FACTOR,
) -> dict[str, tuple[float, float, str]]

```

that returns {name: (min_value, max_value, "Pa")} for each input field with min_value = min(field) * safety_factor if the min is negative else min(field) / safety_factor, and analogously for max. The bounds must enclose the actual field values strictly so that schema-v1's guard G-2 (every cell in [min_value, max_value]) passes. **Do not** hardcode example bounds.

6. Top-level wrapper:

```

def build_stress_safs(
    mesh_paths: list[Path],
    out_path: Path,
    *,
    SHmax: float, Shmin: float, Sv: float, SHmax_az_deg: float,
    depth_model: str = "constant",
    SHmax_grad: float = 0.0, Shmin_grad: float = 0.0,
    Sv_grad: float = 0.0,
    dx_m: float = DEFAULT_GRID_DX_M,
    pad_m: float = DEFAULT_GRID_PAD_M,
    mesh_tag: str = "safs_fault_box_nwcut",
) -> None

```


that:

1. Reads every mesh in mesh_paths (the 500 m / 1000 m / 2000 m bulk VTUs from code_meshing/) and computes the union bbox.
 2. Calls build_uniform_utm_grid.
 3. Calls evaluate_stress_field_on_grid.
 4. Calls derive_field_bounds.
 5. Calls sidecar.write_sidecar(out_path, x, y, z, fields, attrs={"schema_version": SCHEMA_VERSION, "crs": CANONICAL_CRS, "units": CANONICAL_UNITS, "z_positive": CANONICAL_Z_POSITIVE, "source": ",".join(m.name for m in mesh_paths), "source_crs": "EPSG:32611", "mesh_tag": mesh_tag}, field_bounds={...}).
7. CLI mirroring build_velocity_cvmh.py:

```
python build_stress_safs.py
--meshes <vtu1> <vtu2> ...
--out DATA_PROJECTED_ONFAULTSTRESS/stress_safs.h5
--SHmax 113.0 --Shmin 49.0 --Sv 45.0 --SHmax-az 23.0
[--depth-model {constant,lithostatic_sv}]
[--SHmax-grad 0.0 --Shmin-grad 0.0 --Sv-grad 0.0]
[--dx 1000.0 --pad 2000.0]
[--mesh-tag safs_fault_box_nwcut]
```

Interfaces

The public API is build_stress_safs(...) and the CLI. The sidecar is loadable by any schema-v1 reader, including the existing DataField3D C++ class — Phase 6 imposes no additional constraints on the file.

Edge Cases to Handle

- Mesh bbox unioned across resolutions: pad once on the union, not on each mesh — keeps the sidecar generation idempotent across reruns.
- All gradients zero with depth_model="lithostatic_sv": warn (same as Phase 3 §Edge Cases).
- out_path.parent does not exist: created via sidecar.write_sidecar (which already does this — see sidecar.py:78). Do not re-mkdir here; doing so duplicates a defensive sibling in the writer.

Acceptance Criteria

- ☐ Running `python build_stress_safs.py --meshes ../code_meshing/safs_fault_box_nwcut_500m ../code_meshing/safs_fault_box_nwcut_1000m_bulk.vtu --out ../data_projected_onfaultstress/stress_safs.h5` produces a sidecar that h5dump -A reports with the required schema-v1 attributes (schema_version, crs, units, z_positive).
- ☐ `python -c "import h5py; f=h5py.File('stress_safs.h5'); print(list(f['fields']))"` lists exactly the six sigma_* names.
- ☐ Loading the same file via the C++ DataField3D("...", "sigma_xx") ctor in a unit test (Phase 6 stub) succeeds — does not abort — with no Phase-6 code changes.
- ☐ The sidecar's grid bbox strictly encloses the 500 m mesh bbox by at least pad_m on every face.

Dependencies

- Depends on: Phase 3 (the σ^0 field builder), Phase 1 (bbox).
- Required by: Phase 6 (the C++ consumer reads this file).

Phase 6 — C++ consumer: StressField3D, ProjectStress, ComputeSAFSParams

Goal

After Phase 6 the SEAS runtime loads `stress_safs.h5` and uses it to populate per-DOF pre-stress for the quasi-dynamic and dynamic-rupture solvers. The seam matches the velocity pipeline: a sidecar reader backs a per-coefficient Coefficient, which a projector pushes onto either a bulk H1 space (for visualisation / future static-solve) or into the existing `FaultGeometry::tau_pre_ / sigma_n` slots (for production runs).

Files to Create

- `io/stress_field_3d.hpp` — bulk 6-component sidecar reader.
- `io/stress_field_coefficient.hpp` — `mfem::VectorCoefficient` (6 components) backed by six `DataField3D` instances; a thin wrapper.

Files to Modify

- `io/field_coefficient.hpp` — add `FieldProjector::ProjectStress` (bulk, six GFs) and `FieldProjector::ProjectFaultPreStress` (fault, per-DOF) static methods. Mirror the existing `ProjectVelocity` signature (`field_coefficient.hpp:143–150`).
- `io/field_coefficient.cpp` — implementations.
- `fault/fault_geometry.hpp` — add a parallel `ComputeSAFSParams(const StressField3D& field, ...)` next to `ComputeBP5Params()` (`fault_geometry.hpp:703–757`) that populates `tau_pre_` and a new `sigma_n_per_dof_` member array from the sidecar instead of the analytic `bp5_params_.tau0_vec / .sigma_n`.
- `fault/rate_state_fault.hpp` — extend the per-DOF prestress consumption path (currently `tau0_ = params_.tau0()` at lines 255 / 691) to optionally read from `FaultGeometry::tau_pre_per_dof()` and `sigma_n_per_dof()` when SAFS mode is on (new bool flag on the operator).
- `drivers/seas_driver.cpp` — wire a new TOML option `[stress]` with `sidecar = "..."` and a `use_sidecar` boolean, defaulting to false. When `use_sidecar = true`, load the sidecar, call `FaultGeometry::ComputeSAFSParams`, and pass the resulting per-DOF `tau/sigma_n` into the operator.
- `config/seas_config.hpp`, `seas_config_parser.hpp`, `seas_config_bridge.hpp` — add the `[stress]` section to the TOML schema.

Detailed Requirements

1. `StressField3D` — header-only class in `mfem::seas`:

```
class StressField3D
{
public:
    /// Load all six tensor components from a schema-v1 sidecar.
    /// Each component is a separate DataField3D; the ctor instantiates
    /// them in order (sigma_xx, sigma_yy, sigma_zz, sigma_xy,
    /// sigma_yz, sigma_xz) and shares the schema-version /
    /// CRS / bbox checks with DataField3D.
    StressField3D(const std::string& sidecar_path,
                  OOBPolicy oob = OOBPolicy::Abort);

    /// Evaluate the symmetric stress tensor at (x, y, z) in canonical
    /// CRS. Returns a (3, 3) dense matrix in Pa. Compression negative.
    mfem::DenseMatrix Evaluate(real_t x, real_t y, real_t z) const;
```

```

    /// Access the underlying six DataField3D instances by index 0..5
    /// in the order {xx, yy, zz, xy, yz, xz}.
    const DataField3D& Field(int component_index) const;

    const std::array<real_t, 6>& BBox() const;
    bool ContainsBBox(real_t xmin, real_t xmax,
                     real_t ymin, real_t ymax,
                     real_t zmin, real_t zmax,
                     real_t eps = 0.0) const;

    /// Match the InterpMode propagation pattern from DataField3D
    /// (data_field_3d.hpp:88-90). Sets all six underlying readers'
    /// InterpMode in lock-step.
    void SetInterpMode(InterpMode mode);
};

```

Implementation: construct six DataField3D members {"sigma_xx", ..., "sigma_xz"}. Evaluate(x, y, z) returns:

```

DenseMatrix S(3);
const real_t xx = sigma_xx_.Evaluate(x, y, z);
const real_t yy = sigma_yy_.Evaluate(x, y, z);
const real_t zz = sigma_zz_.Evaluate(x, y, z);
const real_t xy = sigma_xy_.Evaluate(x, y, z);
const real_t yz = sigma_yz_.Evaluate(x, y, z);
const real_t xz = sigma_xz_.Evaluate(x, y, z);
S(0,0) = xx; S(1,1) = yy; S(2,2) = zz;
S(0,1) = xy; S(1,0) = xy;
S(0,2) = xz; S(2,0) = xz;
S(1,2) = yz; S(2,1) = yz;
return S;

```

BBox() returns the intersection of all six component bboxes — they are identical by construction (same grid), but the intersection is the schema-conformant value.

2. StressFieldCoefficient (header-only):

```

class StressFieldCoefficient : public mfem::VectorCoefficient
{
public:
    /// 6-component vector coefficient: returns ( $\sigma_{xx}$ ,  $\sigma_{yy}$ ,  $\sigma_{zz}$ ,
    ///  $\sigma_{xy}$ ,  $\sigma_{yz}$ ,  $\sigma_{xz}$ ) at the evaluation point. This matches the
    /// canonical Voigt ordering expected by the static-equilibration
    /// plan §1 (sigma_csm VectorCoefficient).
    StressFieldCoefficient(const StressField3D& field,
                          real_t scale = 1.0,
                          real_t offset = 0.0)
        : VectorCoefficient(6, field_(field), scale_(scale),
                           offset_(offset)) {}

    void Eval(mfem::Vector& v, mfem::ElementTransformation& T,
              const mfem::IntegrationPoint& ip) override;
};

```

Eval calls T.Transform(ip, x) and then six DataField3D::Evaluate calls in the Voigt order, applying the shared scale / offset.

3. FieldProjector::ProjectStress — analog of ProjectVelocity:

```

struct StressFields
{
    std::shared_ptr<mfem::ParGridFunction> sigma_xx;
    std::shared_ptr<mfem::ParGridFunction> sigma_yy;
    std::shared_ptr<mfem::ParGridFunction> sigma_zz;
    std::shared_ptr<mfem::ParGridFunction> sigma_xy;
    std::shared_ptr<mfem::ParGridFunction> sigma_yz;
    std::shared_ptr<mfem::ParGridFunction> sigma_xz;
    real_t min_sigma_xx, max_sigma_xx, ..., max_sigma_xz;    // six pairs
};

static StressFields ProjectStress(
    const std::string&          sidecar_path,
    mfem::ParFiniteElementSpace& target_fes,
    InterpMode interp = InterpMode::Trilinear);

```

Behaviour: six independent Project(...) calls (one per component), each on the same target_fes. Each call increments call_count_ exactly once (so the one-time-load contract is six, not one — but it is a single sidecar load thanks to StressField3D pre-loading every component at construction; we project the already-in-memory fields rather than re-reading HDF5 six times). The min/max bounds are forwarded from the sidecar's per-field declarations.

4. FieldProjector::ProjectFaultPreStress — the new sign-convention boundary, single source of truth:

```

// Project a sidecar stress tensor onto the per-DOF fault basis
// to produce SEAS-internal pre-stress slots:
//   sigma_n_per_dof(i) = -n_i^T σ(x_i) n_i      (compression positive)
//   tau_pre_per_dof(2*i)   = t1_i^T σ(x_i) n_i  (dip component)
//   tau_pre_per_dof(2*i+1) = t2_i^T σ(x_i) n_i  (strike component)
// where (t1, t2, n) is the canonical SEAS fault-local frame
// (CLAUDE.md "fault-local tangent frame" rule: t1 = dip, t2 = strike).
//
// SIGN CONVENTION BOUNDARY: σ(x) from the sidecar is geomechanics
// (compression negative); the emitted sigma_n_per_dof is
// SEAS-internal (compression POSITIVE). This is the only sign
// flip in the runtime pipeline.
static void ProjectFaultPreStress(
    const StressField3D&          field,
    const FaultGeometry&          fault_geom,
    mfem::Vector&                sigma_n_per_dof,    // (num_fault_dofs,)
    mfem::Vector&                tau_pre_per_dof,    // (2 * num_fault_dofs,)
    real_t                        P_p_pa = 0.0,
    real_t                        P_p_grad_pa_per_m = 0.0);

```

The implementation loops over $i = 0 \dots \text{num_fault_dofs} - 1$, fetches $(x_i, t1_i, t2_i, n_i)$ from FaultGeometry (existing accessors: coords_x2_, coords_x3_ plus the fault basis stored in fault_basis_q_ — Phase 6.A must surface the per-DOF basis if it is not already accessible), and computes the three components above. Pore pressure is added: $\text{sigma_n_per_dof}(i) = -n_i^T \sigma - (P_{p_pa} + P_{p_grad_pa_per_m} * \max(0.0, -z_i))$.

5. FaultGeometry::ComputeSAFSPParams — parallel slot to ComputeBP5Params:

```

// SAFS-mode pre-stress initialisation: same outputs as
// ComputeBP5Params (a_values_, eta_values_, dc_values_, tau_pre_,

```

```

/// V_init_vec_) but with `tau_pre_` and the new `sigma_n_per_dof_`
/// sourced from a sidecar. `a`, `Dc`, `eta`, and `V_init_vec_` keep
/// their existing analytic spatial dependence (BP5 a(x2, x3),
/// Dc(x2, x3), etc.). Note: SAFS-specific a/Dc spatial functions
/// are out of scope here; the user can plug in the production
/// SAFS-params analytic forms when ready.
void ComputeSAFSParams(const StressField3D& field,
                      real_t P_p_pa = 0.0,
                      real_t P_p_grad_pa_per_m = 0.0);

```

New private member: Vector `sigma_n_per_dof_`; of size `num_fault_dofs_`. New public const accessor: `const Vector& sigma_n_per_dof() const { return sigma_n_per_dof_; }`.

6. RateStateFault0operator consumer side:

- Add a new private member `const Vector* sigma_n_per_dof_ = nullptr;` and `bool safes_mode_ = false;`.
- In the existing initialisation block at `rate_state_fault.hpp:255` (`tau0_ = params_.tau0()`), guard: `if (!safes_mode_) tau0_ = params_.tau0();` and use the per-DOF fault-geometry pre-stress vector when SAFS mode is active.
- Same for the BP2 path at line 691.
- Inside the inner per-DOF loop (lines 267, 390, 806, 872 — the `tau = tau0_ + traction(i)` sites), replace with: `real_t tau_i = safes_mode_ ? tau_pre_per_dof_>operator()(i) : tau0_;` `tau = tau_i + traction(i);`
- For the normal stress, similar: when `safes_mode_`, the friction solver receives `sigma_n = sigma_n_per_dof_(i)` per DOF instead of the scalar `params_.sigma_n`. Audit *every* call to `sigma_n` in `dieterich_ruina.hpp` and `rate_state_fault.hpp` under the `safes_mode_` branch — the user-feedback memory `feedback_complete_sign_sites.md` mandates a complete audit of ALL occurrences before changing the dispatch.

7. TOML schema extension. Add to `config/seas_config.hpp`:

```

struct StressConfig
{
    bool use_sidecar = false;
    std::string sidecar_path;
    real_t P_p_pa = 0.0;
    real_t P_p_grad_pa_per_m = 0.0;
};

```

Add `StressConfig stress;` to `SeasConfig`. Parser updates in `seas_config_parser.hpp`. Bridge updates in `seas_config_bridge.hpp`. Driver: when `stress.use_sidecar`, instantiate `StressField3D`, call `FaultGeometry::ComputeSAFSParams`, mark the operator `safes_mode_ = true`, attach the per-DOF vectors.

Edge Cases to Handle

- Sidecar mesh bbox does not contain the SAFS fault DOFs: the per-DOF projector aborts via `DataField3D::Evaluate` (interpolation- only contract). This is the desired behaviour; the Phase 5 grid padding is what prevents it from triggering in practice.
- `use_sidecar = true` but no sidecar path: parser aborts with a clear message.
- `use_sidecar = false`: every existing BP5 / BP2 / TPV102 / TPV205 code path remains untouched. The new behaviour is **strictly opt-in**.
- Per-DOF normal stress is negative or zero (rare for SAF geometry but possible in the upper few hundred metres where σ_v is small): emit a warning the first time and clamp via `sigma_n_per_dof(i) = max(sigma_n_per_dof(i), 1.0e6)` (1 MPa floor) only when the value is < 1 MPa. **This clamp is the only numeric default**; document it in the docstring of `ProjectFaultPreStress`. Make the

clamp configurable via a `min_sigma_n_pa` argument that defaults to 0 (no clamp) so the user must opt in.

Acceptance Criteria

- ☐ `StressField3D("stress_safs.h5")` constructs successfully against the Phase-5 output and exposes a (3, 3) symmetric tensor at a sample interior point that matches the H&Z demo_safod σ^0 values to 1e-3 Pa.
- ☐ `FieldProjector::ProjectStress` returns six `ParGridFunctions` whose pointwise values at a sample of cell centroids match the sidecar values to within the trilinear interpolation noise (≤ 1 Pa for a constant field).
- ☐ `FieldProjector::ProjectFaultPreStress` produces `sigma_n_per_dof` whose median equals `-median(sigma_n_total)` from the Phase 4 JSON summary (sign-flipped), to within 1e-3 MPa.
- ☐ `FieldProjector::ProjectFaultPreStress` produces `tau_pre_per_dof(2*i+1)` (strike) whose median matches the `tau_strike` median from the Phase 4 summary to within 1e-3 MPa.
- ☐ Running the existing BP5 verification driver with `--config bp5_example.toml` (which has `stress.use_sidecar = false` by construction) produces bit-exact output relative to the pre-Phase-6 baseline. (The `safs_mode_guard` must compile out cleanly when not in use.)
- ☐ Running a new minimal SAFS TOML (`drivers/safs_smoke.toml`, added in Phase 6) with `stress.use_sidecar = true` and a homogeneous H&Z σ^0 sidecar reaches `t = 1 yr` without nucleating an event (sanity smoke; should be stable under the equilibrium assumption).

Dependencies

- Depends on: Phase 5 (sidecar exists).
 - Required by: Phase 7.
-

Phase 7 — seas_project_stress_to_mesh verification driver

Goal

After Phase 7, a user can visually verify the stress projection in ParaView before launching a long SAFS simulation, exactly as `project_velocity_to_mesh` already does for velocity. The driver loads a mesh + a stress sidecar, projects all six tensor components onto an $H^1(p)$ space, and writes a `.pvd` + per-rank `.vtu` set.

Files to Create

- `drivers/project_stress_to_mesh.cpp` — copy `project_velocity_to_mesh.cpp` and swap `ProjectVelocity` for `ProjectStress`; six `PointData` fields instead of five.
- `tests/unit/test_project_stress_to_mesh.cpp` — drives the driver via subprocess on a tiny synthetic sidecar + mesh; verifies the `pvd` / `vtu` files exist and round-trip-load.

Files to Modify

- `Makefile` — add a `seas_project_stress_to_mesh` target mirroring the existing `seas_project_velocity_to_mesh` target.
- `tests/Makefile` (if present) — add the new unit test to the test suite.

Detailed Requirements

1. CLI matches `project_velocity_to_mesh.cpp:46–95`:

```

seas_project_stress_to_mesh
--mesh      safs_fault_box_nwcut_2000m.msh
--sidecar   stress_safs.h5
--out       projected_stress_2000m
[--order P]
[--interp {trilinear,catmull-rom}]
[--ascii | --binary]

```

Defaults match the velocity driver: `--order 2, --interp trilinear, --binary`.

2. Output: `<out>/<out>.pvd` referencing per-rank `.vtu` files with six PointData scalar fields `{sigma_xx, sigma_yy, sigma_zz, sigma_xy, sigma_yz, sigma_xz}` in Pa.
3. Pre-flight: mesh bbox vs sidecar bbox via `StressField3D::ContainsBBox`. Abort cleanly on failure with the same message format as `FieldProjector::Project`.

Edge Cases to Handle

- Sidecar missing fields: surfaced by the `StressField3D` ctor (each `DataField3D` re-checks).
- `--out` path collides with an existing file: overwrite (matches `project_velocity_to_mesh.cpp`).

Acceptance Criteria

- ☐ `seas_project_stress_to_mesh --mesh ... --sidecar stress_safs.h5 --out projected_stress_2000m` produces a `.pvd` readable in ParaView with six scalar PointData fields, each visualisable as a colour map.
- ☐ Round-trip test: load the resulting VTU back via `meshio` (the Phase 7 unit test does this when `--ascii`), and assert the per-field mean equals the sidecar's per-field mean to 1 part in $1e-9$.
- ☐ The driver runs in under 10 s on the 2000 m mesh, single rank.

Dependencies

- Depends on: Phase 6.
- Required by: nothing.

Testing Strategy

Test file

`code_preprocess/test_project_to_fault_stress.py`, mirroring `test_nw_cut_strip.py` (lines 1–40): `pytest` layout, `sys.path` insert to enable sibling imports, `TestPhase1 / TestPhase2 / ...` class groupings, R-numbered regression tests for any bugfix that lands during implementation.

Per-phase tests

Phase 1 (mesh I/O) - `test_load_fault_vtu_real_data`: loads the real 2000 m fault VTU; asserts triangle count = 2685, bbox matches probed values. - `test_triangle_geometry_unit`: synthetic 3-4-5 right triangle in the XY plane; asserts area = 6.0, normal = $\pm \hat{z}$, centroid = (mean of vertices). Run for 100 random orientations: $lnormal = 1$ to $1e-15$. - `test_tet_geometry_unit`: synthetic reference tet with vertices at the standard simplex; asserts volume = $1/6$, centroid = (0.25, 0.25, 0.25).

Phase 2 (fault basis) - `test_basis_matches_HZ_for_vertical_NW_strike`: synthetic vertical triangle with normal along H&Z's $\hat{n}$ (strike=314, dip=90); assert `per_triangle_basis_raw`

returns $\hat{s}$, $\hat{d}$ matching H&Z to $1e-12$. - `test_basis_orthonormal`: 1000 random triangles, no degenerate cases; $\text{dot}(\hat{s}, \hat{d})$, $\text{dot}(\hat{s}, \hat{n})$, $\text{dot}(\hat{d}, \hat{n})$ all $< 1e-12$ and norms = $1 \pm 1e-12$. - `test_horizontal_triangle_marks_degenerate`: triangle with normal along $\hat{z}$; assert `degen[0] == True` and `strikes[0]` is NaN. - `test_harmonise_idempotent`: random normals; `harmonise(harmonise(x)) == harmonise(x)`. - `test_harmonise_flips_to_majority`: half normals flipped a priori; after harmonisation all dot products with reference > 0 . - `test_harmonise_pca_fallback`: pass `fault_strike_azimuth_hint=None`; on planar synthetic mesh, the resulting normal direction agrees with the analytic plane normal to within $1e-12$.

Phase 3 (stress rotation) - `test_resolve_traction_against_HZ_safod`: SAFOD inputs, single vertical triangle aligned with H&Z's $\hat{s}$ / $\hat{d}$; assert $\sigma_{n_eff} \approx -89$ MPa, $\mu_{apparent} \approx 0.24$, both to $1e-3$ absolute. This is the primary cross-check against the H&Z scalar path and serves as the *numerical regression anchor* for the whole module. - `test_vectorised_equals_scalar`: 100 random triangles, random symmetric σ^0 ; vectorised result vs. 100 calls to H&Z `resolve_traction`, element-wise diff $< 1e-12$. - `test_trace_invariance`: random $(\hat{s}, \hat{d}, \hat{n})$ frames and random symmetric σ ; assert $\sigma_{\hat{n}} + \hat{s}^T \sigma \hat{s} + \hat{d}^T \sigma \hat{d} \approx \text{tr}(\sigma)$ to within $1e-9$. - `test_no_shear_on_principal_face`: σ^0 diagonal with all three distinct eigenvalues, normal aligned with one eigenvector; assert $\tau_{strike} \approx \tau_{dip} \approx 0$ ($< 1e-12$ MPa). - `test_depth_dependent_lithostatic_sv`: 5 sample depths, non-zero gradient; assert σ_{zz} changes linearly with depth and $\sigma_{xx} / \sigma_{yy}$ follow rotation about $\hat{z}$ exactly. - `test_nonsymmetric_sigma_raises`: deliberately non-symmetric input; expect `ValueError`.

Phase 4 (writers + CLI) - `test_write_fault_vtu_roundtrip`: write a synthetic 5-triangle fault VTU; reload with `meshio.read`; assert every expected cell-data field is present with the right shape and dtype. - `test_summary_json_schema`: write a summary; load with `json.load`; assert all top-level keys present and stats has entries for every output field. - `test_cli_print_info_no_write`: invoke `--print-info` via subprocess on the real 2000 m fault VTU; assert no new files appear in `data_projected_onfaultstress/`. - `test_cli_single_file_smoke`: invoke the script on the real 2000 m fault VTU into a tempdir; assert all three expected files appear and pass `meshio.read / json.load`. **This test must run in the pythonenv conda env**; gate with `pytest.importorskip("meshio")`. - `test_vtu_units_metadata_carried`: write a fault VTU, reload, assert the units cell-data attribute is "MPa" for every stress field (or, if `meshio` drops it, assert the JSON summary's units key exists and equals "MPa"). - `test_per_field_minmax_match_summary`: stats reported in the JSON summary must match `np.min / np.max / np.median` of the corresponding VTU cell-data array exactly (no rounding drift). - R-001-style regression slots reserved for issues caught during implementation review.

Phase 5 (sidecar writer) - `test_build_uniform_utm_grid_monotone`: random bbox, random pad, random dx; assert returned x/y/z are strictly monotone increasing. - `test_build_uniform_utm_grid_contains_mesh`: pass the real 500 m mesh bbox; assert sidecar grid bbox encloses it by $\geq$ pad on every face. - `test_evaluate_stress_field_constant_case`: constant depth_model; assert every (N_x, N_y, N_z) cell holds the same six values and those values match H&Z `build_bulk_stress_tensor` outputs to $1e-9$ Pa (after MPa $\rightarrow$ Pa conversion). - `test_evaluate_stress_field_lithostatic_sv`: assert σ_{zz} changes monotonically with depth and $\sigma_{xx} / \sigma_{yy}$ follow the same horizontal rotation at every depth slice. - `test_derive_field_bounds_encloses_data`: every cell value lies strictly inside the derived `[min_value, max_value]`. - `test_no_nan_in_emitted_fields`: every field array has no NaN before passing to `write_sidecar`. - `test_sidecar_schema_v1_conformance`: write a sidecar; reopen with `h5py`; assert `attrs["schema_version"] == "data_projection_v1"`, `attrs["crs"] == "EPSG:32611"`, `attrs["units"] == "m"`, `attrs["z_positive"] == "elevation"`, `created_at` is ISO-8601, every field has `units`, `min_value`, `max_value` attributes, every dataset shape is (N_x, N_y, N_z) , every grid axis is strictly monotone increasing. (This is the defence-in-depth pair of the C++ reader's startup checks.) - `test_sidecar_field_set_is_stress`: assert `list(f["fields"].keys()) == sorted(["sigma_xx", "sigma_yy", "sigma_zz", "sigma_xy", "sigma_yz", "sigma_xz"])` exactly. - `test_sidecar_units_are_Pa`: every `/fields/<name>/units` attribute equals "Pa". - `test_sidecar_reloads_in_DataField3D_compatible_layout`: open the sidecar via the `sidecar.py:read_sidecar_attrs` helper and pass every required attribute check the C++ reader runs (cross-check on the Python side avoids a slow C++ rebuild for an attribute drift bug; the C++ side rechecks

at load anyway). - test_sidecar_roundtrip_evaluator: pick 100 random in-bbox query points, evaluate σ^0 analytically from build_bulk_stress_tensor vs trilinear-interpolated from the sidecar grid; for a constant σ^0 case the difference must be $< 1e-9$ Pa (trilinear of constant is exact). For a lithostatic_sv depth-varying case with a grid step of dx_m, the difference must be $< 0.5 * dx_m * grad_pa_per_m$ per component (linear interpolation error of a linear function is zero at grid points and at-most-grid-step elsewhere). - test_build_stress_safs_idempotent: run build_stress_safs twice with the same args; the two output files are byte-identical except for the created_at attribute. - test_cli_invokes_h5py: gate with pytest.importorskip("h5py"); invoke build_stress_safs.py via subprocess; verify the produced file passes the schema check.

Phase 6 (C++ consumer)

The Phase 6 unit tests live as new files under tests/unit/, matching the style of test_data_field_3d.cpp (custom TEST_ASSERT / TEST_NEAR macros, no gtest dependency).

- test_stress_field_3d_load_schema_v1.cpp: load a synthetic schema-v1 stress sidecar (created by the test in setUp); verify the six DataField3D members are reachable, the bbox is the intersection of the six per-component bboxes, and Evaluate(x, y, z) returns a symmetric (3, 3) tensor whose components match the synthetic sidecar exactly at grid corners.
- test_stress_field_3d_evaluate_symmetric.cpp: 1024 random in-bbox points; assert $S(0, 1) == S(1, 0)$, $S(0, 2) == S(2, 0)$, $S(1, 2) == S(2, 1)$ to machine precision.
- test_stress_field_3d_ctor_aborts_on_wrong_schema.cpp: synthetic sidecar with schema_version = "data_projection_v2"; ctor must abort (use the existing MFEM abort-trap pattern from test_data_field_3d.cpp:257-272).
- test_stress_field_3d_ctor_aborts_on_missing_component.cpp: synthetic sidecar with only five of the six expected fields; ctor must abort with a precise message naming the missing field.
- test_stress_field_3d_ctor_aborts_on_nan.cpp: like test_data_field_3d.cpp:T-3-3 but for the stress reader.
- test_stress_field_3d_interp_mode_lockstep.cpp: call SetInterpMode(CatmullRom); assert all six underlying components report CatmullRom.
- test_field_projector_project_stress_bulk.cpp: build a small serial mesh, write a synthetic constant- σ^0 sidecar, project, assert every ParGridFunction's pointwise value at a sample of DOFs equals the sidecar constant to within $1e-12$ Pa.
- test_field_projector_project_stress_call_count.cpp: verify the call_count_advance: ResetCallCount() then ProjectStress, expect CallCount() == 6 (one per component).
- test_field_projector_project_fault_prestress_basic.cpp: synthetic vertical strike-slip fault triangle; constant σ^0 matching SAFOD; call ProjectFaultPreStress; assert the per-DOF outputs match the Phase 4 Python tau_strike / sigma_n_eff for the same triangle to $1e-3$ Pa. **This is the Python↔C++ parity anchor.**
- test_field_projector_project_fault_prestress_sign_flip.cpp: feed in a compression-negative σ^0 (geomechanics); assert the emitted sigma_n_per_dof values are **positive** (SEAS internal compression-positive convention). This guards the single sign-flip boundary documented in the Constraints section.
- test_field_projector_project_fault_prestress_pore_pressure.cpp: call with P_p_pa = 16e6; assert sigma_n_per_dof(i) is reduced by 16 MPa relative to the P_p_pa = 0 case, at every DOF.
- test_field_projector_project_fault_prestress_pore_pressure_grad.cpp: call with P_p_grad_pa_per_m = $9.8e3 * 1e3$ (hydrostatic $\rho_w = 1000 \text{ kg/m}^3$); assert linear scaling with depth at every fault DOF.
- test_field_projector_project_fault_prestress_min_sigma_n_clamp.cpp: σ^0 field producing negative σ_n at one DOF; call with min_sigma_n_pa = 1e6; assert that DOF receives sigma_n_per_dof = 1e6 and all other DOFs are unchanged.
- test_compute_safs_params_matches_bp5_baseline.cpp: build a synthetic σ^0 sidecar whose value at every fault DOF equals the BP5 analytic tau0_vec / sigma_n; call ComputeSAFSParams and compare tau_pre_ / sigma_n_per_dof_ against the values ComputeBP5Params would produce.

Match within 1e-6 Pa per component. **This is the BP5↔SAFS parity anchor.**

- `test_compute_safs_params_size_matches_num_dofs.cpp`: `tau_pre_.Size() == 2 * num_fault_dofs_` and `sigma_n_per_dof_.Size() == num_fault_dofs_`.
- `test_compute_safs_params_no_nan.cpp`: run on the real 2000 m mesh
 - `real stress_safs.h5`; assert no NaN in either output vector.
- `test_rate_state_fault_safs_mode_dispatch.cpp`: build a minimal fault operator, set `safs_mode_ = true`, plug in per-DOF vectors; assert that subsequent ODE residual calls use the per-DOF values rather than the scalar `params_.tau0() / params_.sigma_n`. (Inspect the resulting traction and back-solve which σ_n value the friction solver received.)
- `test_safs_mode_off_is_bit_exact_to_pre_phase6.cpp`: build the same operator with `safs_mode_ = false`; run one ODE residual; assert byte-exact equality to a snapshot recorded against the pre-Phase-6 baseline (golden vector serialised once during Phase 6 test authoring).
- `test_seas_config_parsing_stress_section.cpp`: TOML round-trip; the `[stress]` section with `use_sidecar = true`, `sidecar = "..."`, `P_p_pa = 16e6` survives parse → bridge → operator-level state.
- `test_seas_config_default_stress_off.cpp`: TOML without a `[stress]` block produces `stress.use_sidecar = false`. Defends the “strictly opt-in” rule.
- `test_safs_smoke_runs_one_step.cpp`: TOML `safs_smoke.toml` with `use_sidecar = true` and a constant- σ^0 sidecar; advance the operator one step; assert it does not abort and $V_{\max} < V_{\text{ini}} * 1e6$ (no immediate runaway).

Phase 7 (verification driver) - `test_project_stress_to_mesh_cli_smoke.cpp`: `subprocess-invoke seas_project_stress_to_mesh --mesh <tiny.msh> --sidecar <synth.h5> --out <tmp/out> --ascii`; assert `<tmp/out>/out.pvd` exists and references at least one `*.vtu`. Reload the VTU via `meshio` in a helper Python script (run via `subprocess.run([sys.executable, ...])`), assert the six `PointData` fields are present. - `test_project_stress_to_mesh_mean_matches_sidecar.cpp`: same setup; assert the per-field mean of each `PointData` array equals the sidecar’s per-field mean (constant field) to 1 part in 1e-9. - `test_project_stress_to_mesh_bbox_failure.cpp`: drive with a sidecar whose `bbox` does NOT enclose the mesh; the driver must exit with a non-zero code and emit the expected `ProjectStress: mesh bbox is NOT contained` message.

Reference data

- The `H&Z demo_safod()` numerical output is the **gold standard** for the SAFOD parameter case. Capture its current console-printed values once during Phase 3 test authoring, embed them as constants in the test file with a comment citing the H&Z file line numbers, and use them as the cross-check anchor.
- The Phase 4 JSON summary for the 2000 m mesh + SAFOD parameters is the **Python↔C++ parity anchor**. Generate it once at the end of Phase 4, freeze its per-field median values into a constant `tests/data/safs_2km_safod_medians.json` file checked into the repo, and reference it from the Phase 6 parity tests.

Risk Assessment

Numerical risks

- **Normal-orientation drift across a curved fault.** If `harmonise_normal_orientation` flips a sub-cluster of triangles incorrectly, the fault basis will have a sign inversion in that region, causing `tau_strike` to invert sign locally. **Detection:** the JSON summary’s `tau_strike.min` and `tau_strike.max` should be same-signed for a regional-stress projection onto a single fault; a sign mix indicates the harmonisation failed. **Mitigation:** the PCA fallback + explicit strike hint + idempotency test cover this. Manual ParaView spot-check of `normal_vec` on the 2000 m mesh is the final QA.

- **Near-vertical sliver triangles** from the NW-cut pipeline can have $|n_z|$ very close to 1 (effectively horizontal). The `NEAR_HORIZONTAL_NZ_TOL` threshold needs to be loose enough not to flag legitimate sub-vertical triangles. We log `n_degenerate`; if it is $> 0.1\%$ of the mesh, investigate before treating output as trustworthy.
- **Symmetry in σ^0** : small floating-point asymmetry from the H&Z rotation can trip the `ValueError`. Use `atol=1e-9`.

Codebase risks

- **VTU cell-data lifetime in meshio.** meshio's VTU writer used to drop `field_data` silently. Confirm with a round-trip test (`write -> read -> compare`) at Phase 4 start; if it drops, fall back to writing the convention string into the JSON summary only.
- **Coordinate frame assumption.** The mesh is currently UTM Zone 11N metres. If a future mesh re-projects to a different CRS or to a rotated local frame, the H&Z assumption (`x = east, y = north, z = up`) breaks. **Mitigation:** the `bbox` check in Phase 1 (with warning) flags the obvious case; document the assumption prominently in the module docstring and the JSON summary.

Scope risks

- **The CSM static-equilibration plan defines a fuller, MFEM-side pipeline** that needs σ^0 as a `VectorCoefficient` for downstream L2 projection (`data_stress_frictionlaw/community_stress_model/PLAN_c` §1). Phase 5 of this plan produces the sidecar that plan's Phase 1 consumes; we **must not** start implementing the L2-equilibration solve here.
- **Multi-fault future case.** The current pipeline produces a single fault group (`fault, tag 101`). When multi-fault becomes real, the module's `extract_cells_by_physical` will need a list-input form. Mark this as a follow-up in the module docstring; do not pre-build the multi-group form now (it would invite premature abstraction and the user's feedback memory warns against this).

Phase-6 / Phase-7 specific risks

- **Sign-convention boundary drift.** `ProjectFaultPreStress` is the single sign-flip site. If any downstream consumer (e.g. a future developer adding a new friction model) accidentally re-flips, the shear convention will invert and produce the v8-style positive feedback runaway documented in `CLAUDE.md`. **Mitigation:** the parity test against the Phase 4 JSON medians (the `test_field_projector_project_fault_prestress_basic.cpp` test) detects this; the docstring of `ProjectFaultPreStress` lists every sign convention crossing as a literal table.
- **One-time-load contract.** `ProjectVelocity` advances `call_count_` by three (one per field). `ProjectStress` advances by six. The existing T-4-7 unit test in `test_data_field_3d.cpp` will break if it asserts an exact count; audit that test before running Phase 6 (see `feedback_complete_sign_sites.md` — audit ALL occurrences).
- **bp5_params_.sigma_n is currently a single scalar.** Several consumer code paths read it directly (`rate_state_fault.hpp:806, 872, dieterich_ruina.hpp`). The Phase 6 audit must list every such site and decide for each whether the SAFS-mode dispatch is needed. The user-feedback memory `feedback_complete_sign_sites.md` governs: list ALL sites in the docstring of `ComputeSAFSPParams` before changing any one of them.
- **TOML schema additions can break existing configs.** The `[stress]` block must default to “off” (`use_sidecar = false`) and the parser must accept TOMLs that omit the block entirely. The `test_seas_config_default_stress_off.cpp` test enforces this.
- **C++ unit test build cost.** The Phase 6 / 7 tests require rebuilding the SEAS library. Schedule the test runs so each topic-area's unit tests can be invoked in isolation (via the `make test-stress-projection` target, added in Phase 6).

Implementation-process risks

- **Easy to silently bake hardcoded numerics** (SHmax = 113 etc.). The user's `feedback_no_hardcoded_numbers.md` memory governs: *every* numerical default must come from a named CLI flag whose default originates from the H&Z `demo_safod` block, captured in one named constant near the top of the module.

Files Summary

Python (Phases 1–5)

File	Status	Purpose
<code>code_preprocess/project_to_fault_stress.py</code>	new	Library + CLI (Phases 1–4)
<code>code_preprocess/test_project_fault_stress.py</code>	new	pytest suite (Phases 1–4)
<code>code_preprocess/build_stress_safs.py</code>	new	schema-v1 sidecar writer (Phase 5)
<code>code_preprocess/test_build_stress_safs.py</code>	new	pytest suite (Phase 5)
<code>code_preprocess/PLAN_onfaultstress.md</code>	new	This document
<code>code_preprocess/clean_freestress.sh</code>	unchanged	Reference style
<code>code_preprocess/nw_cut_strip.py</code>	unchanged	Reference style
<code>code_preprocess/msh_to_vtu.py</code>	unchanged	Reference style; we read its outputs
<code>code_preprocess/ts_to_stl.py</code>	unchanged	Reference style
<code>code_preprocess/data_projected_onfaultstress.py</code>	unchanged	Imported as library (Phase 5)
<code>data_stress_frictionlaw/regional_stress_projection/hickman_block.py</code>	unchanged	Imported as library

Python output artefacts

File	Phase	Purpose
<code>data_projected_onfaultstress4<lc>m/<base>_fault_stress.vtu</code>	4	Fault ParaView (debug/QA)
<code>data_projected_onfaultstress4<lc>m/<base>_bulk_stress.vtu</code>	4	Bulk ParaView (debug/QA)
<code>data_projected_onfaultstress4<lc>m/<base>_summary.json</code>	4	Parameter + stats summary
<code>data_projected_onfaultstress5stress_safs.h5</code>	5	Canonical sidecar (production input)

C++ (Phases 6–7)

File	Status	Purpose
<code>io/stress_field_3d.hpp</code>	new	Bulk 6-component schema-v1 reader
<code>io/stress_field_coefficient.hpp</code>	new	<code>mfem::VectorCoefficient</code> adapter
<code>io/field_coefficient.hpp</code>	modified	Add <code>ProjectStress</code> , <code>ProjectFaultPreStress</code>
<code>io/field_coefficient.cpp</code>	modified	Implementations
<code>io/data_field_3d.hpp</code>	unchanged	Reused as the per-component reader
<code>fault/fault_geometry.hpp</code>	modified	Add <code>ComputeSAFSParams</code> , <code>sigma_n_per_dof</code>

File	Status	Purpose
fault/rate_state_fault.hpp	modified	safs_mode_ dispatch on per-DOF prestress
friction/dieterich_ruina.hpp	audited	every sigma_n site listed; modified only where audit requires
config/seas_config.hpp	modified	Add StressConfig
config/seas_config_parser.hpp	modified	TOML [stress] block parse
config/seas_config_bridge.hpp	modified	Pass StressConfig to driver
drivers/seas_driver.cpp	modified	Wire [stress] opt-in path
drivers/project_stress_to_mesh.cpp	new	Visualisation driver (Phase 7)
drivers/safs_smoke.toml	new	Minimal SAFS-mode smoke config
Makefile	modified	seas_project_stress_to_mesh target
tests/unit/test_stress_fieldnew*.cpp	new	Phase 6 reader tests
tests/unit/test_field_projectnewproject_stress*.cpp	new	Phase 6 projector tests
tests/unit/test_compute_safsnrams*.cpp	new	Phase 6 fault-geometry tests
tests/unit/test_rate_state_new_safs_mode*.cpp	new	Phase 6 operator dispatch tests
tests/unit/test_seas_config_newstress*.cpp	new	Phase 6 TOML tests
tests/unit/test_project_stressnewto_mesh*.cpp	new	Phase 7 driver tests
tests/data/safs_2km_safod_mednewns.json	new	Frozen parity-anchor medians